33

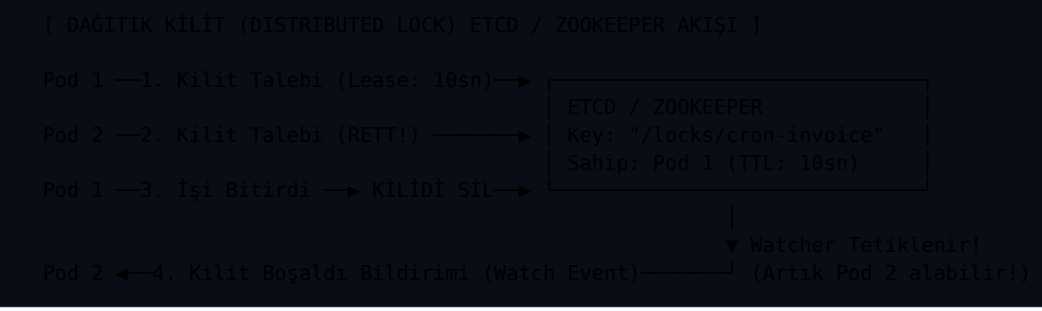
BÖLÜM: DAĞITIK SISTEM TEMELLERI VE EŞGÜDÜM | MODULE: DISTRIBUTED SYSTEMS FUNDAMENTALS, CONSENSUS & COORDINATION

Dağıtık Eşgüdüm Servisi: Apache ZooKeeper ve etcd (Dağıtık Kilit & Lider)

Distributed Coordination: ZooKeeper & etcd (Distributed Locks & Leader)

📐 Sistem Mimarisi & Veri Akış Şeması

📐 Architecture & Data Flow Diagram



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

10 odalı bir pansiyonda tek bir ortak banyo olduğunu düşünün. Banyonun kapısına 'Dolu / Boş' yazan bir kilit asarsınız (Dağıtık Kilit - Distributed Lock). İçeri giren kişi kapıyı kilitler; dışarıdakiler kapının açılmasını bekler (Watcher). Eğer içerideki kişi banyoda bayılırsa kapı sonsuza kadar kilitli kalmasın diye kilit 10 dakika sonra otomatik açılır (Lease / TTL). İşte `etcd` ve `ZooKeeper`, binlerce sunucunun aynı anda tek bir kaynağa hücum edip birbirini ezmesini engelleyen bu güvenilir kilit servisidir!

🇬🇧 ENGLISH (ELI5)

Picture a boarding house with 10 rooms and a single shared bathroom. On the door hangs a physical latch (Distributed Lock). When someone enters, they slide the latch; others queue outside watching the door (Watcher). To prevent the door locking forever if someone falls asleep inside, the latch springs open automatically after 10 minutes (Lease / TTL). `etcd` and `ZooKeeper` serve as that centralized, bulletproof lock coordinator for thousands of server pods!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

etcd ve Apache ZooKeeper; hiyerarşik anahtar-değer ağacı (znodes), güçlü tutarlılık (Raft / Zab), geçici düğümler (Ephemeral nodes / Leases) ve olay dinleyicileri (Watchers) sunan dağıtık eşgüdüm servisleridir. Kullanım Alanları: 1) Dağıtık Kilit (Distributed Lock): Tek bir cron görevinin sadece tek bir podda çalışmasını sağlamak. 2) Lider Seçimi (Leader Election): Kümedeki podlardan biri geçici bir düğüm (`/leader`) oluşturur; çökerse Lease düşer ve Watcher diğer podları uyandırarak yeni lideri seçtirir. 3) Konfigürasyon Yönetimi: Sistem konfigürasyonunu merkezi olarak tutar; konfigürasyon değiştiğinde tüm mikroservisler anında Watcher ile güncellenir.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

etcd and ZooKeeper provide strongly consistent, linearizable key-value hierarchies backed by Raft/ZAB consensus. Core primitives: 1) Distributed Locking: A worker acquires a key bound to a time-limited Lease (heartbeat keep-alive). If the worker crashes, the lease expires and purges the key automatically, avoiding deadlocks. 2) Leader Election: Nodes race to claim an ephemeral key (`/leader/worker-A`). The winner leads; competitors register Watchers on the key path to react instantly to failures. 3) Dynamic Configuration: Centralizes runtime flags; changes trigger push notifications across thousands of microservice Watcher streams within milliseconds.

⚡ Modern Java 25 Uygulaması (Loom & Records)

⚡ Java 25 Implementation

```java
// Java 25: Dağıtık Kilit (Distributed Lock) Mantığı Simülasyonu
import java.util.concurrent.*;

public class DistributedLockSim {
    private String lockOwner = null;
    private long leaseExpiration = 0;

    public synchronized boolean acquireLock(String workerId, long leaseDurationMillis) {
        long now = System.currentTimeMillis();
        // Kilit boş mu veya süresi dolmuş mu?
        if (lockOwner == null || now > leaseExpiration) {
            lockOwner = workerId;
            leaseExpiration = now + leaseDurationMillis;
            IO.println("🔒 Kilit Alındı: " + workerId + " (Süre: " + leaseDurationMillis + "ms)");
            return true;
        }
        return false; // Kilit meşgul!
    }

    public synchronized void releaseLock(String workerId) {
        if (workerId.equals(lockOwner)) {
            lockOwner = null;
            IO.println("🔓 Kilit Bırakıldı: " + workerId);
        }
    }
}

void main() {
    var locker = new DistributedLockSim();
    boolean pod1 = locker.acquireLock("Pod-1", 5000);
    boolean pod2 = locker.acquireLock("Pod-2", 5000); // Başarısız olmalı!

    IO.println("Pod-1 Başarılı mı? " + pod1);
    IO.println("Pod-2 Başarılı mı? " + pod2);
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

etcd veya ZooKeeper'ı devasa veriler (büyük JSON'lar, resimler) saklamak için KULLANMAYIN! Bu sistemlerin bellek sınırı ve mimarisi yalnızca küçük metadata ve kilitler (birkaç kilobayt) içindir; büyük veriler Raft konsensüs kütüğünü kilitler.

⚠️ Interview Trap & Bottleneck Alert

🇬🇧 INTERVIEW TRAP & BOTTLENECK

Never store bulky payloads inside etcd or ZooKeeper. They are strictly designed for micro-metadata and consensus coordination (under 1MB per value). Storing heavy state degrades Raft commit replication across nodes.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- etcd ve ZooKeeper, dağıtık sistemlerde kilit ve lider seçiminin güvenilir adresidir.
- Lease (Kira / TTL) mekanizması, düğüm çöktüğünde kilitlerin sonsuza kadar kilitli kalmasını (Deadlock) önler.
- Watcher API, veritabanını sürekli sorgulamak (polling) yerine olay tabanlı anlık bildirim sağlar.

🎯 Key Takeaways for the Interview

- etcd and ZooKeeper serve as the authoritative standard for distributed locks and elections.
- Time-bound leases prevent distributed deadlocks by releasing locks upon node failure.
- Watcher streams replace expensive database polling with event-driven notifications.